

Spring Data JPA Internals

A Deep Dive Tutorial - Java Backend Series

Chapter 06 - Persistence Layer Mechanics

A JPA / Hibernate / Spring Data layering	Three layers, one abstraction
B Repository proxy creation at runtime	JdkDynamicAopProxy + SimpleJpaRepository
C Derived query method parsing	PartTree to JPQL under the hood
D @Query with JPQL	Named and positional parameters
E Entity lifecycle states	Transient, Persistent, Detached, Removed
F Persistence Context and first-level cache	StatefulPersistenceContext internals
G N+1 problem	Detection and 3 fixes
H Pagination	Pageable, Page, LIMIT/OFFSET
I @Transactional on repositories	What Spring Data adds by default
J Custom repository fragments	When and how to implement

A JPA vs Hibernate vs Spring Data

The layering explained clearly

Step 1: Plain English Explanation

JPA (Jakarta Persistence API) is a specification - a set of interfaces and rules written in a document (JSR 338). It defines what a persistence provider must do but contains no runnable code. Hibernate is the most popular implementation of that specification. It provides the actual classes that map Java objects to database rows. Spring Data JPA sits on top of both: it reads your repository interface and generates boilerplate code at startup so you never have to write EntityManager calls yourself.

Step 2: Real-World Analogy

Think of JPA as the USB standard - it defines the plug shape and voltage. Hibernate is a specific USB cable manufacturer. Spring Data JPA is the device driver that auto-detects the cable and wires it into your operating system, so you just call 'copy file' without worrying which brand of cable is plugged in.

Step 3: Minimal Working Java Code

```
// JPA annotation (javax.persistence / jakarta.persistence)
@Entity
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String email;
}

// Spring Data JPA repository -- zero implementation needed
public interface UserRepository extends JpaRepository<User, Long> { }
```

```
// Usage in a service
@Service
public class UserService {
    private final UserRepository repo;
    public User findById(Long id) { return repo.findById(id).orElseThrow(); }
}
```

Step 4: Diagram

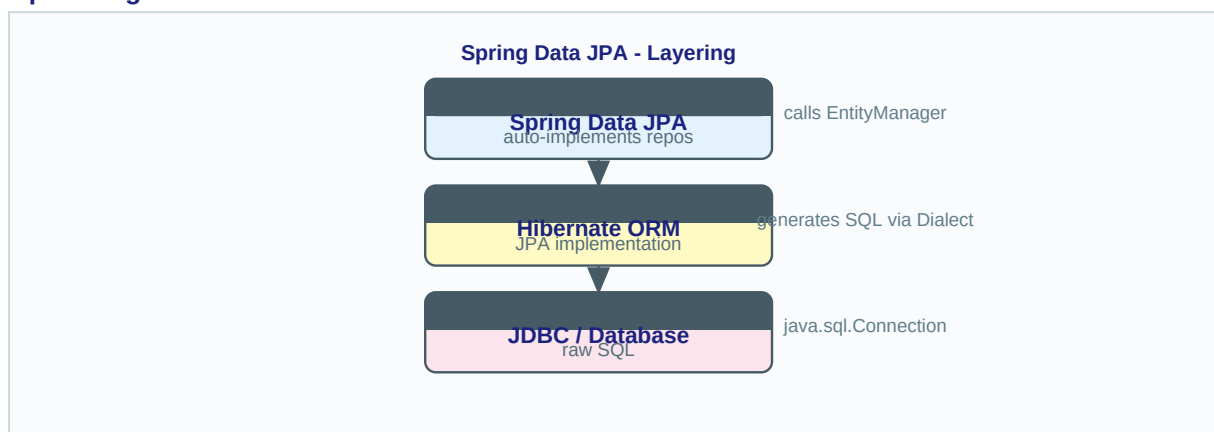


Figure A: The three-layer stack from your code to the database.

Step 5: Behind the Scenes

At startup Spring scans for interfaces extending Repository. For each it delegates to RepositoryFactorySupport.getRepository(), which creates a JpaRepositoryFactory. That factory

instantiates a `SimpleJpaRepository` - a concrete class that calls `EntityManager` internally. It then wraps that instance in a `JdkDynamicAopProxy` so that Spring's `@Transactional` advice, query routing, and auditing can intercept every method call before it reaches `SimpleJpaRepository`.

Step 6: Realistic Practical Example

A Spring Boot project with `spring-boot-starter-data-jpa` on the classpath. You declare `UserRepository`, inject it into `UserService`, call `repo.save(user)`, and the generated proxy opens a transaction, calls `EntityManager.persist(user)`, flushes to the DB, and commits - all without a single line of implementation code.

Step 7: Common Mistakes and Misconceptions

- Mistake: Thinking Spring Data replaces Hibernate. It does not - it still uses Hibernate under the hood.
 - Mistake: Bypassing Spring Data and using `EntityManager` directly in the repository layer. This loses transaction management.
 - Misconception: JPA annotations like `@Entity` come from Hibernate. They come from the `jakarta.persistence` package (the JPA spec).
 - Misconception: Spring Data JPA generates SQL. It generates JPQL; Hibernate translates JPQL to SQL using the configured Dialect.
-

B Repository Proxy Creation at Runtime

JdkDynamicAopProxy wraps SimpleJpaRepository

Step 1: Plain English Explanation

You write only an interface. Spring Data somehow turns it into a working bean with save, findById, delete, and all your custom methods. The magic happens during the application context refresh: Spring scans for repository interfaces, builds an implementation object (SimpleJpaRepository), wraps it in a Java dynamic proxy, and registers the proxy as a singleton bean. When you call `repo.findById(1L)` you are calling the proxy, not SimpleJpaRepository directly.

Step 2: Real-World Analogy

Imagine ordering a custom suit. You hand the tailor only a measurement card (your interface). The tailor (Spring) builds the actual suit (SimpleJpaRepository), puts it inside a garment bag with pockets for alterations (the proxy), and delivers the bag. When you reach into the bag for the suit, an attendant checks your ID and logs the transaction before handing it over (AOP advice).

Step 3: Minimal Working Java Code

```
// Your interface
public interface ProductRepository extends JpaRepository<Product, Long> {
    List<Product> findByCategory(String category);
}

// Spring Boot auto-configuration bootstraps this at startup:
// 1. JpaRepositoryAutoConfiguration triggers
// 2. EnableJpaRepositoryRegistrar scans for Repository sub-interfaces
// 3. RepositoryFactorySupport.getRepository(ProductRepository.class) runs
// 4. Result: a JdkDynamicAopProxy bean named "productRepository"

@Service
public class Catalog {
    @Autowired
    private ProductRepository products; // actually the proxy
}
```

Step 4: Diagram

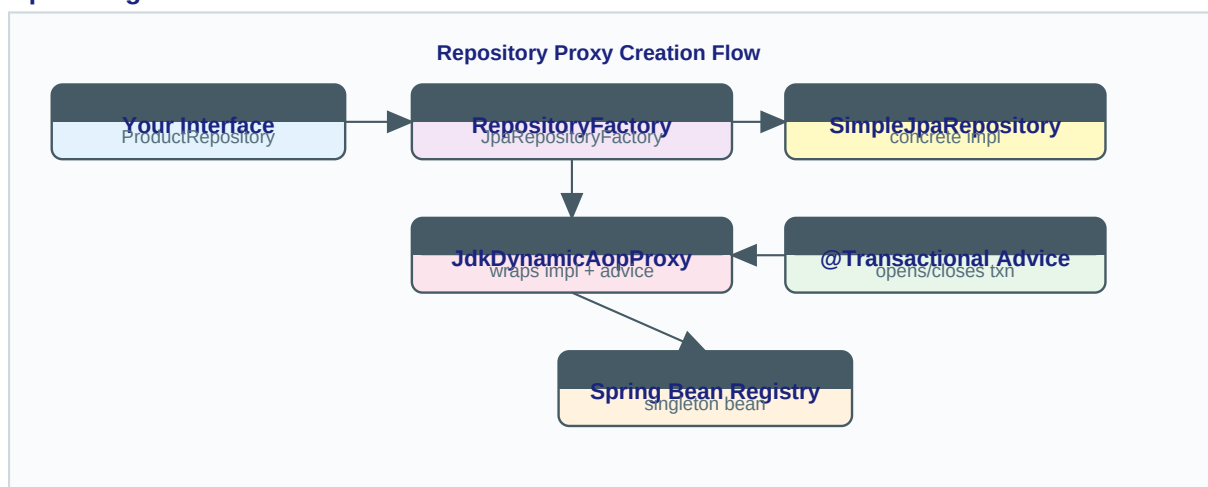


Figure B: Spring Data proxy creation and wiring at startup.

Step 5: Behind the Scenes

RepositoryFactorySupport.getRepository() is the key method. It calls getTargetRepository() which returns a new SimpleJpaRepository(domainClass, entityManager). It then calls getRepositoryInformation() to resolve all method metadata. Finally ProxyFactory.getProxy() produces a JdkDynamicAopProxy. Every method invocation hits QueryExecutorMethodInterceptor.invoke(), which routes the call to either a CRUD method on SimpleJpaRepository, a derived query, or a @Query method.

Step 6: Realistic Practical Example

Add spring.jpa.show-sql=true and a debug log on org.springframework.data.repository. At startup you will see lines like 'Creating proxy for ProductRepository' confirming the proxy is built. At runtime each call logs the generated SQL, proving the proxy intercepts and delegates correctly.

Step 7: Common Mistakes and Misconceptions

- Mistake: Annotating the repository interface with @Repository. Spring Data does it implicitly; adding it again is harmless but redundant.
 - Mistake: Expecting to cast the injected bean to SimpleJpaRepository. You get a Proxy instance; casting will throw ClassCastException.
 - Misconception: The proxy is CGLib-based. For interfaces Spring uses JdkDynamicAopProxy (java.lang.reflect.Proxy), not CGLib.
 - Misconception: Every method call creates a new proxy. The proxy is a singleton; only the EntityManager-scoped objects change per request.
-

C

Derived Query Method Parsing

How findByEmailAndAge becomes JPQL

Step 1: Plain English Explanation

Spring Data reads your method name character by character and converts it into a JPQL query. The method name has three parts: a Subject (find, count, exists, delete), optional keywords (Distinct, Top3), and a Predicate (the conditions). The predicate is split on 'And' and 'Or', and each segment is matched against entity field names. Parameters are bound positionally in the same order they appear in the method signature.

Step 2: Real-World Analogy

A natural-language parser like voice recognition. You say 'find me flights by airline and departure date'. The system breaks it down: action=find, entity=flights, conditions=[airline, departureDate]. It then builds a database query from that structure automatically. Spring Data's parser does the same with Java method names.

Step 3: Minimal Working Java Code

```
// Method declaration
List<User> findByEmailAndAgeGreaterThan(String email, int age);

// Spring Data parses this as:
// Subject -> findBy (returns List<User>)
// Part 1 -> Email (exact match, column: email)
// Connector-> And
// Part 2 -> Age (field: age)
// Keyword -> GreaterThan (operator: >)
//
// Generated JPQL:
// SELECT u FROM User u
// WHERE u.email = ?1 AND u.age > ?2

// More examples:
Optional<User> findFirstByLastNameOrderByCreatedAtDesc(String lastName);
long countByActiveTrue();
boolean existsByEmail(String email);
void deleteByExpiredAtBefore(LocalDateTime threshold);
```

Step 4: Diagram

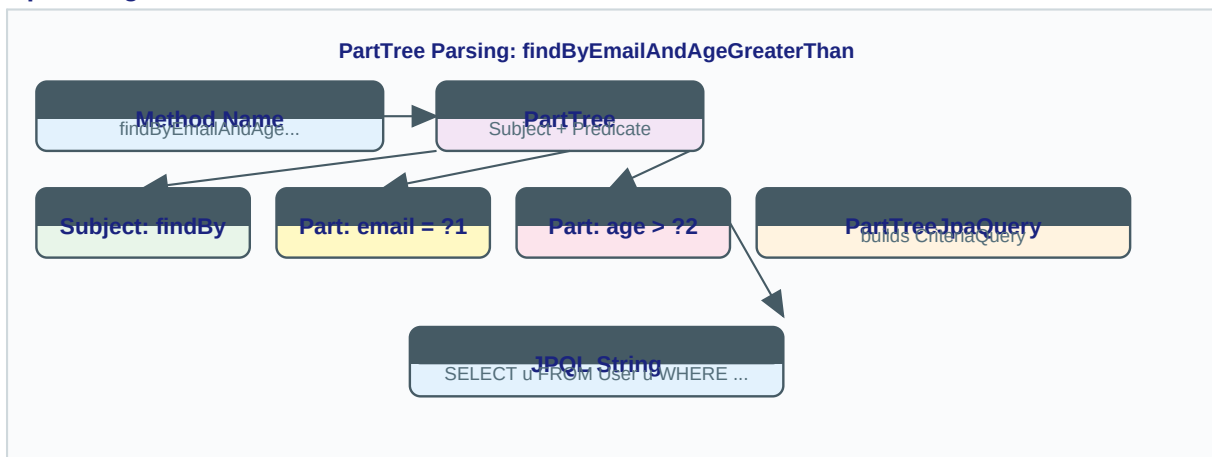


Figure C: PartTree breaks the method name into Subject and Predicate parts.

Step 5: Behind the Scenes

The class `PartTree` (spring-data-commons) tokenises the method name using a regex. It identifies the subject prefix (find, read, get, count, exists, delete) and then splits the remainder on 'And' and 'Or' using `OrPart` and `AndPart`. Each leaf segment is wrapped in a `Part` object that maps to a JPA metamodel attribute and a keyword operator (Contains, StartingWith, GreaterThan, Between, etc.). `PartTreeJpaQuery` then hands the tree to `JpaQueryCreator` which builds a JPA `CriteriaQuery` or a JPQL string depending on which strategy is selected.

Step 6: Realistic Practical Example

```
// Repository
interface OrderRepository extends JpaRepository<Order, Long> {
    List<Order> findByStatusAndCreatedAtBetween(
        OrderStatus status, LocalDateTime from, LocalDateTime to);
    List<Order> findTop5ByCustomerIdOrderByCreatedAtDesc(Long customerId);
}

// Generated JPQL (approximately):
// SELECT o FROM Order o
//   WHERE o.status = ?1
//   AND o.createdAt BETWEEN ?2 AND ?3
//
// SELECT o FROM Order o
//   WHERE o.customerId = ?1
//   ORDER BY o.createdAt DESC
// (with LIMIT 5 added by the dialect)
```

Step 7: Common Mistakes and Misconceptions

- Mistake: Naming a method `findByEmailaddress` when the field is `emailAddress`. Spring cannot find the field and throws `PropertyReferenceException` at startup.
 - Mistake: Using And/Or inside field names (e.g., a field called 'brandOrModel'). Spring tries to split on Or and misparses the name.
 - Misconception: The query is rebuilt on every call. It is parsed once at startup and cached as a `PreparedStatement`.
 - Misconception: Derived queries are slower than `@Query`. They are identical at runtime - both become `PreparedStatements`.
-

D @Query with JPQL

Named parameters, positional parameters

Step 1: Plain English Explanation

@Query lets you write the JPQL (or native SQL) directly on the repository method. JPQL looks like SQL but targets entity class names and field names, not table names and column names. There are two ways to bind parameters: positional (?1, ?2) which bind by argument order, and named (:name) which bind by the @Param annotation on the method parameter. Named parameters are preferred because they survive argument reordering.

Step 2: Real-World Analogy

Positional parameters are like a recipe card that says 'add ingredient 1, then ingredient 2'. If you swap the jars the dish is ruined. Named parameters are like a recipe that says 'add flour: 2 cups, sugar: 1 cup' - you can put the jars in any order and the result is always correct.

Step 3: Minimal Working Java Code

```
public interface InvoiceRepository extends JpaRepository<Invoice, Long> {

    // Positional parameters -- order matters
    @Query("SELECT i FROM Invoice i WHERE i.status = ?1 AND i.amount > ?2")
    List<Invoice> findByStatusAndMinAmount(String status, BigDecimal minAmount);

    // Named parameters -- order does NOT matter
    @Query("SELECT i FROM Invoice i WHERE i.customerId = :cid AND i.dueDate < :today")
    List<Invoice> findOverdue(@Param("cid") Long customerId,
                             @Param("today") LocalDate today);

    // Native SQL -- use nativeQuery = true
    @Query(value = "SELECT * FROM invoices WHERE status = :s LIMIT 10",
           nativeQuery = true)
    List<Invoice> findTop10Native(@Param("s") String status);

    // Modifying query
    @Modifying
    @Query("UPDATE Invoice i SET i.status = :s WHERE i.id = :id")
    int updateStatus(@Param("id") Long id, @Param("s") String status);
}
```

Step 4: Diagram

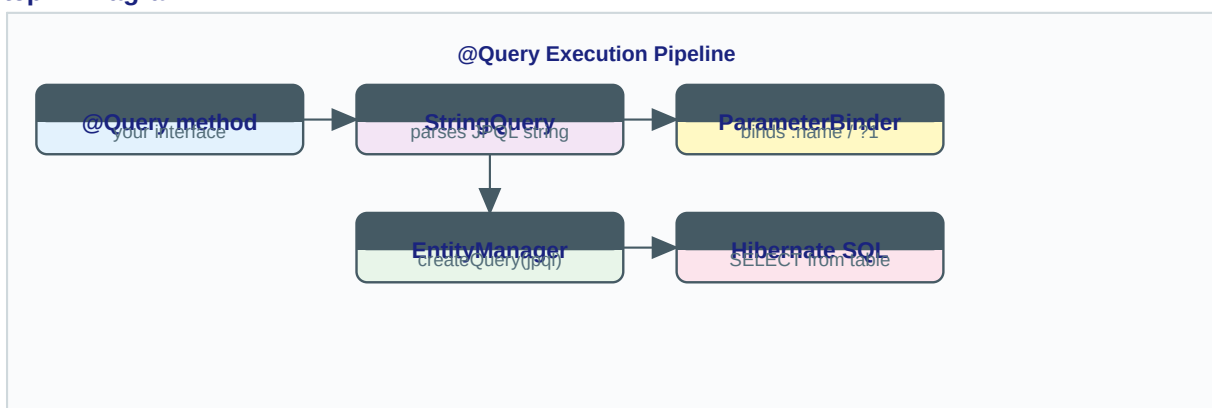


Figure D: @Query string is parsed, parameters bound, then executed via EntityManager.

Step 5: Behind the Scenes

Spring Data wraps the JPQL string in a `StringQuery` object which parses parameter placeholders with a regex. For named parameters it creates `NamedParameterQueryParameterSetter`; for positional it uses `PositionalQueryParameterSetter`. At execution time `AbstractJpaQuery.doExecute()` calls `createQuery()` on the `EntityManager`, then `ParameterBinder` iterates the `ParameterMetadataProvider` list and calls `query.setParameter()` for each argument. The resulting `TypedQuery` is then executed with `getResultList()` or `getSingleResult()`.

Step 6: Realistic Practical Example

```
// Complex report query with DTO projection
@Query("SELECT new com.example.dto.SalesSummary(i.customerId, SUM(i.amount)) " +
    "FROM Invoice i " +
    "WHERE i.createdAt BETWEEN :from AND :to " +
    "GROUP BY i.customerId " +
    "ORDER BY SUM(i.amount) DESC")
List<SalesSummary> monthlySalesSummary(
    @Param("from") LocalDateTime from,
    @Param("to")    LocalDateTime to);
```

Step 7: Common Mistakes and Misconceptions

- Mistake: Writing 'FROM invoices' (table name) in JPQL. Use the entity class name: 'FROM Invoice'.
 - Mistake: Omitting `@Modifying` on UPDATE/DELETE queries. Without it Spring Data executes `getSingleResult()` and throws an exception.
 - Mistake: Forgetting `@Transactional` on `@Modifying` methods. Without a transaction the update is rejected.
 - Misconception: `nativeQuery=true` means no parameter binding. Parameter binding still works with named and positional parameters.
-

E

Entity Lifecycle States

Transient, Persistent, Detached, Removed

Step 1: Plain English Explanation

Every entity object is always in one of four states relative to the Persistence Context. Transient: a plain Java object that JPA knows nothing about. Persistent (Managed): the object is inside the current Persistence Context - changes are automatically synced to the database at flush time. Detached: the Persistence Context that knew about this object has been closed; changes are ignored until you re-attach. Removed: you called `remove()`; the row will be deleted at flush.

Step 2: Real-World Analogy

Think of a Persistence Context as a shared whiteboard in a meeting room. Transient: notes on your private notepad - the room cannot see them. Persistent: notes you have written on the whiteboard - any change is instantly visible to the room. Detached: you ripped your section off the board and walked out - the room no longer tracks your notes. Removed: you erased your section - it will be gone when the meeting ends (flush).

Step 3: Minimal Working Java Code

```
@Transactional
public void demonstrateLifecycle() {
    // TRANSIENT -- not yet known to JPA
    User u = new User();
    u.setEmail("alice@example.com");

    // PERSISTENT -- EntityManager tracks this object
    entityManager.persist(u); // or repo.save(u) for new entity

    // u.id is now set; changes will be flushed automatically
    u.setEmail("alice-updated@example.com"); // no explicit save needed

    // REMOVED -- row deleted at flush
    entityManager.remove(u);
} // transaction commits -> flush -> SQL executed -> context closed

// After the method returns, u is DETACHED
// Calling u.setEmail(...) now has no effect on the database

public void reattach(User detached) {
    // DETACHED -> PERSISTENT again
    User merged = entityManager.merge(detached);
    // merged is the new persistent copy; detached remains detached
}
```

Step 4: Diagram

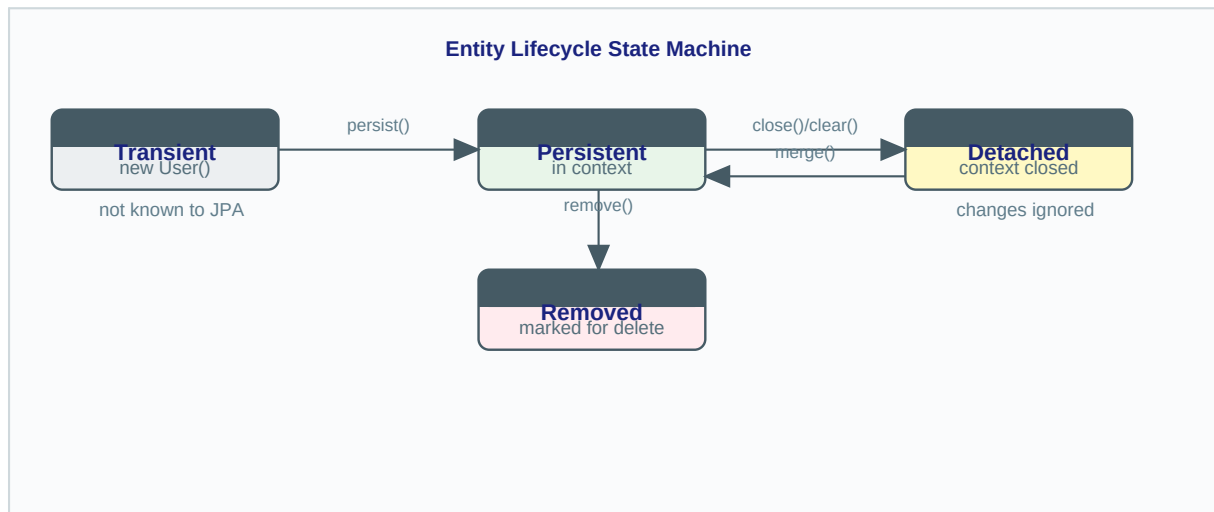


Figure E: Entity state transitions driven by EntityManager operations.

Step 5: Behind the Scenes

Hibernate's `StatefulPersistenceContext` maintains an `entityInstancesByKey` map (an `IdentityMap` keyed by `EntityKey` = entity class + primary key). When `persist()` is called Hibernate adds the entity to this map and to the `EntityEntryContext`, setting the entry status to `Status.MANAGED`. At flush time the dirty checker walks each entry, compares current field values to the snapshot taken at load time, and issues UPDATE SQL for any changed entities. `remove()` sets the status to `Status.DELETED`; `flush()` then executes DELETE. When the Session (Persistence Context) closes, all entries become `Status.DETACHED`.

Step 6: Realistic Practical Example

```
// Classic detached entity bug
@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id) {
    return userRepository.findById(id).orElseThrow();
    // Transaction opened and CLOSED here -- user is DETACHED
}

// If lazy-loaded collections are accessed after this method returns,
// you get LazyInitializationException because the Session is gone.

// Fix: use a DTO, or annotate with @Transactional on the endpoint method,
// or use spring.jpa.open-in-view=true (controversial -- see concept F).
```

Step 7: Common Mistakes and Misconceptions

- Mistake: Calling `repo.save()` on a persistent entity inside a transaction. It is a no-op for updates - dirty checking handles it.
- Mistake: Reusing a detached entity as if it were persistent. You must call `merge()` or retrieve a fresh copy.
- Misconception: `merge()` updates the detached object in-place. It returns a new persistent instance; the argument stays detached.
- Misconception: `remove()` deletes immediately. It only marks the entity; the DELETE SQL runs at the next flush.

F

Persistence Context and First-Level Cache

Identity guarantee, dirty checking

Step 1: Plain English Explanation

The Persistence Context is a short-lived workspace (one per transaction by default) that holds every entity loaded or saved during that transaction. Its first-level cache means: if you load the same row twice in the same transaction you get the exact same Java object (identity guarantee) - Hibernate does not hit the database the second time. Additionally it takes a snapshot of each loaded entity so it can detect changes automatically (dirty checking) and issue UPDATE statements without you calling save().

Step 2: Real-World Analogy

A physical inbox on your desk during a work session. Every document you request is placed in the inbox. If you request the same document twice, the assistant checks the inbox first and hands you the same copy - no trip to the archive room. When the work session ends the assistant compares each document to its original (the snapshot), stamps any changed pages, and files the changes back to the archive.

Step 3: Minimal Working Java Code

```
@Transactional
public void cacheDemo() {
    // First load -- issues SELECT
    User u1 = userRepository.findById(1L).orElseThrow();

    // Second load -- returns SAME object from first-level cache, no SELECT
    User u2 = userRepository.findById(1L).orElseThrow();

    System.out.println(u1 == u2); // true -- identical Java reference

    // Dirty checking -- no repo.save() call needed
    u1.setEmail("new@example.com");
    // At transaction commit: Hibernate detects the change, issues UPDATE
}
```

Step 4: Diagram

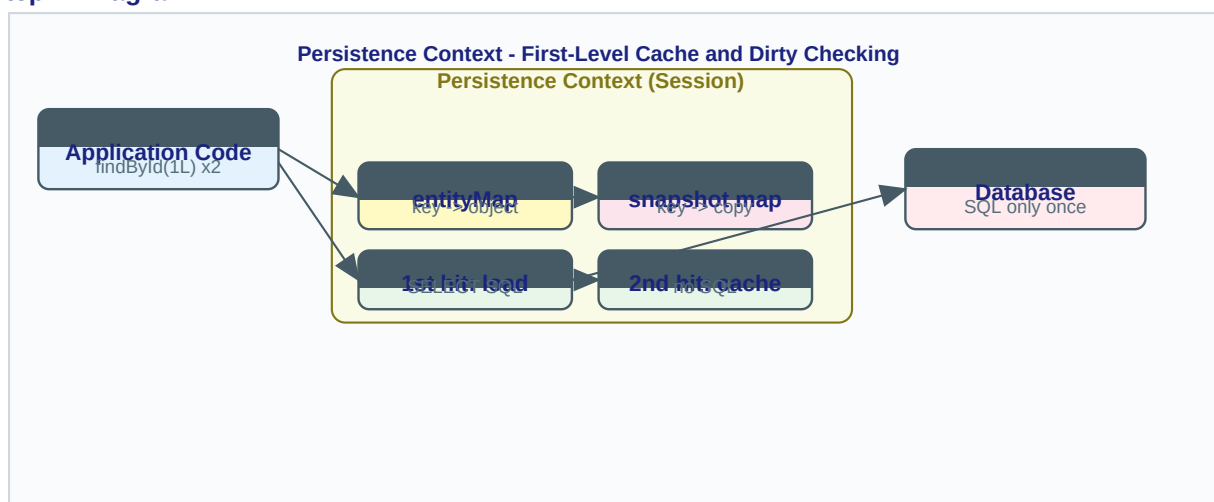


Figure F: Persistence Context caches entities and tracks snapshots for dirty checking.

Step 5: Behind the Scenes

The implementation class is `org.hibernate.engine.internal.StatefulPersistenceContext`. It holds a field called `entitiesByKey` (type `Map`) - this is the first-level cache. When `EntityManager.find()` is called, Hibernate first checks `entitiesByKey`; if found it returns the cached instance immediately. If not, it loads from the database and stores both the live object and a deep copy (the `hydratedState` array stored in `EntityEntry`) as the snapshot. At flush time `DefaultFlushEntityEventListener.dirtyCheck()` walks each `EntityEntry`, calls `TypeHelper.findDirty()`, and compares current state to `hydratedState` field-by-field using the JPA type's `equals` logic. If differences are found an `UPDATE` is scheduled.

Step 6: Realistic Practical Example

```
@Transactional
public void bulkUpdatePrices(List<Long> productIds, BigDecimal discount) {
    for (Long id : productIds) {
        Product p = productRepository.findById(id).orElseThrow();
        p.setPrice(p.getPrice().multiply(BigDecimal.ONE.subtract(discount)));
        // NO repo.save(p) -- dirty checking issues UPDATE at commit
    }
    // WARNING: all products remain in the first-level cache.
    // For large lists (10 000+) this exhausts memory.
    // Fix: call entityManager.flush() + entityManager.clear() every 50 rows.
}
```

Step 7: Common Mistakes and Misconceptions

- Mistake: Expecting the first-level cache to survive multiple transactions. It is transaction-scoped by default (TRANSACTION scope).
 - Mistake: Calling `entityManager.clear()` inside a loop without flushing first. Unflushed changes are lost.
 - Misconception: The second-level cache is the same as the first-level cache. L2 cache (EhCache, Caffeine) is shared across sessions; L1 is per-session only.
 - Misconception: `spring.jpa.open-in-view=true` extends the first-level cache safely. It holds the session open for the entire HTTP request, causing subtle staleness and connection pool exhaustion under load.
-

G

N+1 Problem

Why it happens, detection, 3 fixes

Step 1: Plain English Explanation

The N+1 problem occurs when loading N parent entities triggers N additional SELECT statements - one per parent - to load a lazily-fetched association. If you load 100 orders, Hibernate executes 1 query to get the orders and then 100 more queries to load the Customer for each order - 101 queries total. This is almost always unintentional and can degrade performance by orders of magnitude.

Step 2: Real-World Analogy

Imagine a librarian fetching 100 book records from the catalogue (1 query). For each book she then walks to a different shelf to look up the author biography (100 queries). A smart librarian would prepare a single list of all 100 author IDs and retrieve all biographies in one trip - that is the JOIN FETCH / batch fetch solution.

Step 3: Minimal Working Java Code - Problem and 3 Fixes

```
@Entity
public class Order {
    @ManyToOne(fetch = FetchType.LAZY)    // LAZY is the default for @ManyToOne
    private Customer customer;
}

// --- PROBLEM ---
List<Order> orders = orderRepository.findAll(); // 1 SELECT
for (Order o : orders) {
    System.out.println(o.getCustomer().getName()); // N SELECTs!
}

// --- FIX 1: JOIN FETCH in JPQL ---
@Query("SELECT o FROM Order o JOIN FETCH o.customer")
List<Order> findAllWithCustomer();

// --- FIX 2: @EntityGraph ---
@EntityGraph(attributePaths = {"customer"})
List<Order> findAll(); // override default findAll

// --- FIX 3: Batch fetching ---
// application.properties:
// spring.jpa.properties.hibernate.default_batch_fetch_size=50
// Hibernate issues: SELECT * FROM customers WHERE id IN (?, ?, ?, ..., ?)
// instead of N individual SELECTs
```

Step 4: Diagram



Figure G: N+1 fires one SELECT per entity; JOIN FETCH collapses them into one.

Step 5: Behind the Scenes - Detection

Enable Hibernate statistics by setting `spring.jpa.properties.hibernate.generate_statistics=true`. After a request the log will contain lines like:

```
Session Metrics {
  145234 nanoseconds spent acquiring 1 JDBC connections;
  0 nanoseconds spent releasing 0 JDBC connections;
  1023456 nanoseconds spent preparing 101 JDBC statements;
  101 flushes as part of transactions
}
HikariPool: 101 statements executed

// 101 statements for a findAll() on 100 rows = classic N+1
// Fix applied: drops to 2 statements (1 query + 1 IN batch or join)
```

Also useful: add `p6spy` or `datasource-proxy` to your classpath. Both print every SQL statement with the call stack, making it trivial to identify which repository method is the root cause.

Step 6: Realistic Practical Example

```
// Reporting endpoint that triggered N+1 in production
@GetMapping("/reports/daily-orders")
public List<OrderSummaryDTO> dailyOrders(@RequestParam LocalDate date) {
    List<Order> orders = orderRepository.findByDate(date); // 1 query, 500 rows
    return orders.stream()
        .map(o -> new OrderSummaryDTO(
            o.getId(),
            o.getCustomer().getName(), // N+1 here!
            o.getProduct().getSku())) // another N+1!
        .collect(toList());
}

// Fixed repository method:
@Query("SELECT o FROM Order o " +
    "JOIN FETCH o.customer " +
    "JOIN FETCH o.product " +
    "WHERE o.date = :date")
List<Order> findByDateWithDetails(@Param("date") LocalDate date);
// Result: 1 query instead of 1001
```

Step 7: Common Mistakes and Misconceptions

- Mistake: Using FetchType.EAGER on @ManyToOne thinking it prevents N+1. EAGER without JOIN FETCH still issues N individual SELECTs via secondary queries.
 - Mistake: JOIN FETCH on a @OneToMany collection with pagination. Hibernate returns a Cartesian product and applies pagination in memory - use batch fetch or a subquery instead.
 - Misconception: @EntityGraph is always safer than JOIN FETCH. Both produce a JOIN; @EntityGraph is just a more flexible, annotation-driven way to express it.
 - Misconception: N+1 only happens with @ManyToOne. It also occurs with @OneToOne, @OneToMany, and any lazily-loaded collection.
-

H **Pagination**

Pageable, Page, how LIMIT/OFFSET is generated

Step 1: Plain English Explanation

Pagination lets you retrieve a subset of query results instead of all rows at once. Spring Data's Pageable interface carries page number, page size, and optional sort information. Pass it to any repository method and Spring Data automatically appends LIMIT and OFFSET (or the equivalent for your database) to the generated SQL. The returned Page object includes the content slice, total element count, and total page count.

Step 2: Real-World Analogy

A library catalogue with 10 000 books. Instead of printing all 10 000 entries, the librarian gives you a booklet of 20 entries (one page). The booklet cover says 'Page 3 of 500, showing entries 41-60'. Pageable is the request ('give me page 3, size 20'), Page is the returned booklet.

Step 3: Minimal Working Java Code

```
// Repository -- just add Pageable parameter
public interface ArticleRepository extends JpaRepository<Article, Long> {
    Page<Article> findByPublishedTrue(Pageable pageable);

    // With @Query
    @Query("SELECT a FROM Article a WHERE a.category = :cat ORDER BY a.createdAt DESC")
    Page<Article> findByCategory(@Param("cat") String cat, Pageable pageable);
}

// Service usage
@Service
public class ArticleService {
    public Page<Article> getPage(int page, int size) {
        Pageable pg = PageRequest.of(page, size, Sort.by("createdAt").descending());
        return articleRepository.findByPublishedTrue(pg);
    }
}

// Page object
Page<Article> result = service.getPage(0, 20);
result.getContent();           // List<Article> -- 20 items
result.getTotalElements();     // total matching rows
result.getTotalPages();        // ceil(totalElements / size)
result.hasNext();              // true if more pages
```

Step 4: Diagram

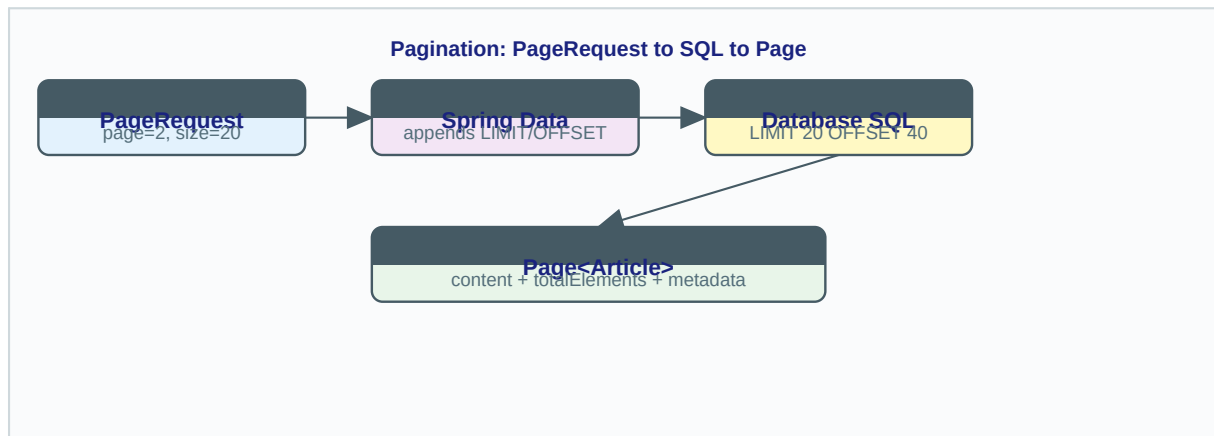


Figure H: PageRequest flows to SQL LIMIT/OFFSET; result wrapped in Page metadata.

Step 5: Behind the Scenes

When Spring Data detects a Pageable parameter it delegates to `PagedExecution.execute()` inside `RepositoryQueryExecution`. This calls `entityManager.createQuery(jpql).setFirstResult(offset).setMaxResults(pageSize)`. Hibernate's dialect translates these into database-specific SQL: PostgreSQL uses LIMIT/OFFSET, SQL Server uses FETCH NEXT n ROWS ONLY, Oracle uses ROWNUM or FETCH FIRST. To populate `Page.totalElements` a separate COUNT query runs automatically unless you return `Slice` (which skips the COUNT) or provide a `countQuery` in `@Query`.

Step 6: Realistic Practical Example

```
// REST controller with pagination
@GetMapping("/products")
public ResponseEntity<Page<ProductDTO>> listProducts(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "20") int size,
    @RequestParam(defaultValue = "name") String sort) {

    Pageable pageable = PageRequest.of(page, size, Sort.by(sort).ascending());
    Page<Product> products = productRepository.findAll(pageable);
    Page<ProductDTO> dtos = products.map(ProductDTO::from);
    return ResponseEntity.ok(dtos);
}

// Slice -- when you do NOT need total count (e.g., infinite scroll)
Slice<Product> slice = productRepository.findByCategory("books",
    PageRequest.of(0, 20));
slice.hasNext(); // only checks if next page exists -- no COUNT query
```

Step 7: Common Mistakes and Misconceptions

- Mistake: Using Page with JOIN FETCH. The COUNT query conflicts with JOIN FETCH and Hibernate emits a warning and applies pagination in-memory.
- Mistake: Passing page -1 or Integer.MAX_VALUE to fetch all. Use `findAll()` with a Sort instead.
- Misconception: Slice is always faster than Page. It is faster only because it skips the COUNT query; the data query is identical.
- Misconception: The default sort is stable. Without an explicit Sort the order is database-dependent and non-deterministic across pages.

I @Transactional on Repositories

What Spring Data adds by default

Step 1: Plain English Explanation

Spring Data JPA marks every method of `SimpleJpaRepository` with `@Transactional`. Read methods (`findById`, `findAll`) get `@Transactional(readOnly=true)` which tells the persistence provider not to track dirty state, improving performance. Write methods (`save`, `delete`) get plain `@Transactional` which opens or joins a read-write transaction. If a caller already has an active transaction the repository joins it; otherwise it opens a new one.

Step 2: Real-World Analogy

A bank teller window (the repository) has default policies for each service: 'balance inquiry' is handled in read-only mode (no ledger changes needed, faster). 'Withdrawal' opens a full transaction. If you are already inside a branch manager session (outer service transaction), the teller joins your session rather than starting a new one.

Step 3: Minimal Working Java Code

```
// What SimpleJpaRepository looks like internally (simplified)
@Repository
@Transactional(readOnly = true)          // class-level default
public class SimpleJpaRepository<T, ID> {

    @Transactional                      // overrides class-level for writes
    public <S extends T> S save(S entity) { ... }

    @Transactional
    public void deleteById(ID id) { ... }

    // findById inherits class-level readOnly = true
    public Optional<T> findById(ID id) { ... }
}

// Your service should ALSO be @Transactional to wrap multiple calls
@Service
@Transactional
public class OrderService {
    public void placeOrder(Order order) {
        orderRepository.save(order);      // joins service transaction
        inventoryRepository.decrement(order); // same transaction
    }
    // both saves commit or roll back together
}
```

Step 4: Diagram

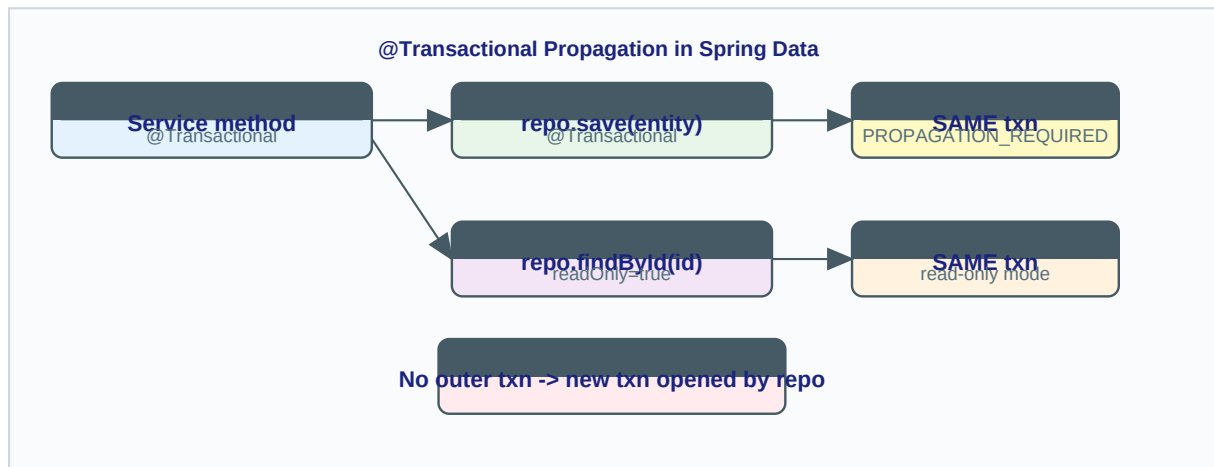


Figure 1: Repository joins the caller's transaction; falls back to a new one if none exists.

Step 5: Behind the Scenes

Spring's TransactionInterceptor sits in the AOP proxy chain around every @Transactional method. When the method is entered it calls PlatformTransactionManager.getTransaction(TransactionDefinition). If a transaction already exists on the thread-local TransactionSynchronizationManager, PROPAGATION_REQUIRED joins it. Otherwise a new DataSourceTransactionManager (or JpaTransactionManager) transaction is opened. readOnly=true hints to the JDBC driver and Hibernate: Hibernate skips dirty-check snapshots, reducing memory; some JDBC drivers route the query to a read replica.

Step 6: Realistic Practical Example

```

// Bug: service NOT @Transactional but repository IS
@Service
public class ReportService {
    public void generateReport(Long orderId) {
        Order o = orderRepository.findById(orderId).orElseThrow();
        o.setReportGeneratedAt(LocalDateTime.now()); // change detected?
        // NO! -- the repository's findById transaction already committed.
        // The entity is now DETACHED. The change is silently lost.
    }
}

// Fix: annotate the service method
@Transactional
public void generateReport(Long orderId) {
    Order o = orderRepository.findById(orderId).orElseThrow();
    o.setReportGeneratedAt(LocalDateTime.now()); // dirty check fires at commit
}
  
```

Step 7: Common Mistakes and Misconceptions

- Mistake: Annotating the repository interface instead of the service with @Transactional. Repositories already have it; you need it on the service to span multiple calls.
- Mistake: Calling a @Transactional method from within the same bean. Spring AOP cannot intercept self-calls; the transaction annotation is ignored.
- Misconception: readOnly=true prevents all writes. It is only a hint - Hibernate and some JDBC drivers respect it; others do not. A write inside readOnly=true can succeed.
- Misconception: @Transactional rolls back on checked exceptions by default. Only RuntimeException triggers rollback unless rollbackFor is specified.

J

Custom Repository Fragments

When and how to implement them

Step 1: Plain English Explanation

Sometimes derived queries and @Query are not enough: you need to call EntityManager directly, use the Criteria API, call a stored procedure, or integrate with a non-JPA data source. Spring Data lets you define a fragment interface and provide an implementation class. Spring merges the fragment implementation with SimpleJpaRepository to produce the final bean. You can add as many fragments as needed.

Step 2: Real-World Analogy

A Swiss Army knife. The standard blade (SimpleJpaRepository) covers most jobs. When you need a corkscrew (custom batch import) or a saw (full-text search via stored proc) you snap in an extension module. The knife's handle (your repository interface) exposes all tools as if they were built-in.

Step 3: Minimal Working Java Code

```
// Step 1: Fragment interface
public interface ProductSearchFragment {
    List<Product> searchByKeywords(List<String> keywords);
}

// Step 2: Implementation class -- MUST be named <Interface>Impl
public class ProductSearchFragmentImpl implements ProductSearchFragment {

    @PersistenceContext
    private EntityManager em;

    @Override
    public List<Product> searchByKeywords(List<String> keywords) {
        CriteriaBuilder cb = em.getCriteriaBuilder();
        CriteriaQuery<Product> cq = cb.createQuery(Product.class);
        Root<Product> root = cq.from(Product.class);
        List<Predicate> preds = keywords.stream()
            .map(k -> cb.like(cb.lower(root.get("description")),
                "%" + k.toLowerCase() + "%"))
            .collect(toList());
        cq.where(cb.and(preds.toArray(new Predicate[0])));
        return em.createQuery(cq).getResultList();
    }
}

// Step 3: Merge into the main repository
public interface ProductRepository
    extends JpaRepository<Product, Long>, ProductSearchFragment {
}

// Usage
List<Product> results = productRepository
    .searchByKeywords(List.of("organic", "fair-trade"));
```

Step 4: Diagram

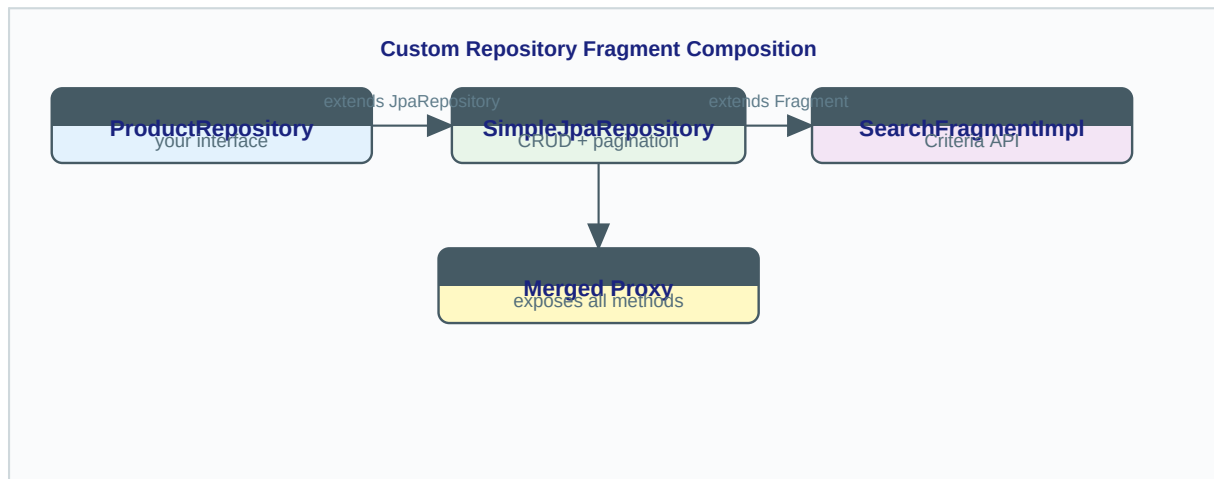


Figure J: Spring merges fragment implementations into one proxy bean.

Step 5: Behind the Scenes

`RepositoryFactorySupport.getRepository()` calls `findRepositoryFragments()` which scans for classes named `Impl` in the same package (or the configured `repositoryImplementationPostfix`, default `'Impl'`). Each fragment is wrapped in an `ImplementationMethodExecutionInterceptor`. The proxy's method dispatcher checks whether the called method belongs to the fragment interface; if so it routes to the fragment implementation instance rather than to `SimpleJpaRepository`. Multiple fragments are composed in declaration order.

Step 6: Realistic Practical Example

```
// Fragment for bulk upsert using Criteria API
public interface BulkUpsertFragment<T> {
    int bulkInsertIgnoreDuplicates(List<T> entities);
}

public class BulkUpsertFragmentImpl implements BulkUpsertFragment<Product> {
    @PersistenceContext EntityManager em;

    @Transactional
    public int bulkInsertIgnoreDuplicates(List<Product> products) {
        int count = 0;
        for (int i = 0; i < products.size(); i++) {
            em.merge(products.get(i));
            if (i % 50 == 0) { em.flush(); em.clear(); }
            count++;
        }
        return count;
    }
}

public interface ProductRepository
    extends JpaRepository<Product, Long>, BulkUpsertFragment<Product> { }
```

Step 7: Common Mistakes and Misconceptions

- Mistake: Naming the implementation class differently from `Impl`. Spring will not find it and throws `NoSuchBeanDefinitionException` at startup.
- Mistake: Putting the `Impl` class in a different package without configuring `repositoryBasePackage`. Scan does not cross package boundaries by default.

- Misconception: Fragment methods override SimpleJpaRepository methods. They do not - if the method signature conflicts, Spring Data throws an ambiguous mapping error at startup.
 - Misconception: You need a fragment for every non-trivial query. In most cases @Query with JPQL or Specifications covers the use case with far less boilerplate.
-

Quick Reference Cheat Sheet

Concept	Key Class / API	Important Detail
Repository proxy	RepositoryFactorySupport, JdkDynamicAopProxy	Created at startup; wraps SimpleJpaRepository
Derived query parsing	PartTree, PartTreeJpaQuery	Parsed once at startup, cached as PreparedStatement
@Query	StringQuery, ParameterBinder	Use named params (:name) over positional (?1)
Entity lifecycle	EntityManager.persist/merge/remove	Persistent entities auto-sync at flush
First-level cache	StatefulPersistenceContext.entitiesByKey	Per-session; identity guarantee within transaction
Dirty checking	EntityEntry.hydratedState, DefaultFlushEntityEventListener	Snapshot compared field-by-field at flush
N+1 fix 1	JOIN FETCH in JPQL	Best for small result sets with known associations
N+1 fix 2	@EntityGraph(attributePaths=...)	Declarative; can be per-method
N+1 fix 3	hibernate.default_batch_fetch_size	Best for large collections; uses IN clause
Pagination	PageRequest.of(), Page, Slice	Slice skips COUNT; use for infinite scroll
@Transactional	TransactionInterceptor, PROPAGATION_REQUIRED	readOnly=true skips dirty-check snapshots
Custom fragment	Impl naming convention	Merged into proxy by RepositoryFactorySupport